

Programmation Linéaire

Corrigé complet — Exercices 1 à 6

July 11, 2026

Contents

1	Maximisation standard	1
1.1	Programme linéaire	1
1.2	Droites de contraintes et région admissible	1
1.3	Optimum graphique	1
1.4	Implémentation PuLP	2
2	Minimisation (région non bornée)	2
2.1	Domaine admissible (ouvert / non borné)	2
2.2	Optimum par la méthode de la droite d'isocoût	3
2.3	Vérification avec PuLP	3
3	Solutions multiples	4
3.1	Sommets et évaluation	4
3.2	Démonstration de l'infinité de solutions	4
3.3	Vérification avec PuLP	4
4	Domaine non borné	5
4.1	Analyse	5
4.2	Erreur renvoyée par PuLP	5
5	Sensibilité	6
5.1	Sommets	6
5.2	Condition de stabilité du sommet (4, 2)	6
5.3	Vérification avec PuLP	6
6	Planification de production multi-sites	7
6.1	Modélisation	7
6.2	Programme linéaire	7
6.3	Implémentation Python (PuLP, variables indexées)	7
6.4	Résultat	8

Exercice 1 Maximisation standard

Une artisane mauritanienne confectionne deux types de tapis : *Standard* (x_1) et *Luxe* (x_2).

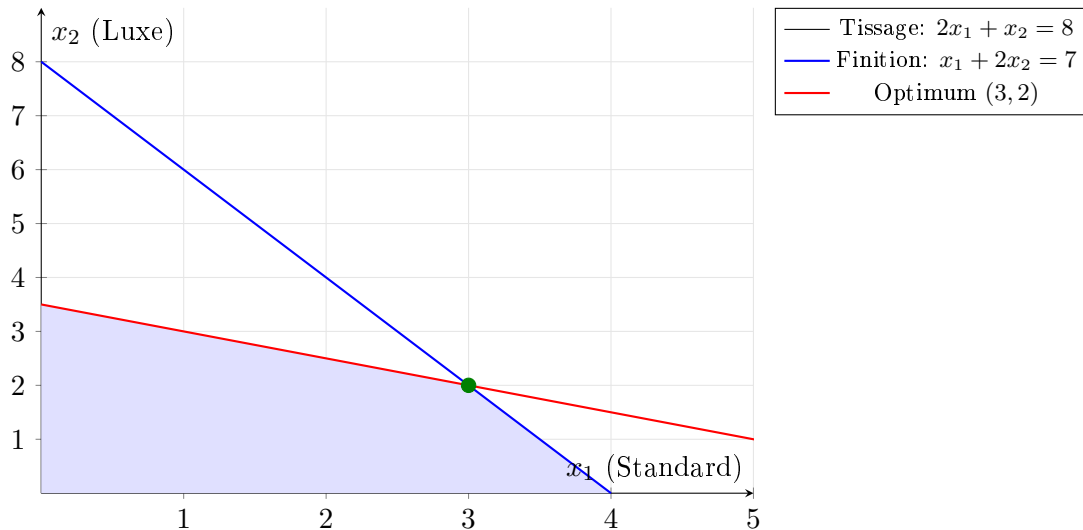
- Standard : 2h tissage, 1h finition, profit 300 MRU.
- Luxe : 1h tissage, 2h finition, profit 400 MRU.
- Disponibilité : 8h (tissage) et 7h (finition).

1.1 Programme linéaire

Variables : x_1 = nombre de tapis Standard, x_2 = nombre de tapis Luxe.

$$\begin{aligned} \max Z &= 300x_1 + 400x_2 \\ \text{sous } \begin{cases} 2x_1 + x_2 \leq 8 & (\text{tissage}) \\ x_1 + 2x_2 \leq 7 & (\text{finition}) \\ x_1, x_2 \geq 0 \end{cases} \end{aligned}$$

1.2 Droites de contraintes et région admissible



1.3 Optimum graphique

Les sommets de la région admissible sont $(0, 0)$, $(4, 0)$, $(3, 2)$, $(0, 3,5)$.

Sommet	$Z = 300x_1 + 400x_2$
$(0, 0)$	0
$(4, 0)$	1200
$(3, 2)$	1700
$(0, 3,5)$	1400

Optimum : $x_1 = 3$, $x_2 = 2$, avec $Z^* = 1700$ MRU. L'artisanne doit produire **3 tapis Standard et 2 tapis Luxe**. Les deux ressources (tissage : 8h, finition : 7h) sont entièrement utilisées à l'optimum, ce qui confirme géométriquement que ce sommet est optimal.

1.4 Implémentation PuLP

```

1  import pulp
2
3  prob = pulp.LpProblem("Maximisation_Profit_Tapis", pulp.LpMaximize)
4
5  x1 = pulp.LpVariable("x1_Standard", lowBound=0)
6  x2 = pulp.LpVariable("x2_Luxe", lowBound=0)
7
8  prob += 300 * x1 + 400 * x2, "Profit_total"
9  prob += 2 * x1 + 1 * x2 <= 8, "Tissage"
10 prob += 1 * x1 + 2 * x2 <= 7, "Finition"
```

```

11
12 prob.solve()
13
14 print("Statut :", pulp.LpStatus[prob.status])
15 print(f"x1 = {x1.varValue}, x2 = {x2.varValue}")
16 print(f"Z = {pulp.value(prob.objective)} MRU")

```

Sortie attendue : Statut: Optimal, $x_1 = 3.0$, $x_2 = 2.0$, $Z = 1700.0$ MRU — confirme exactement la solution graphique.

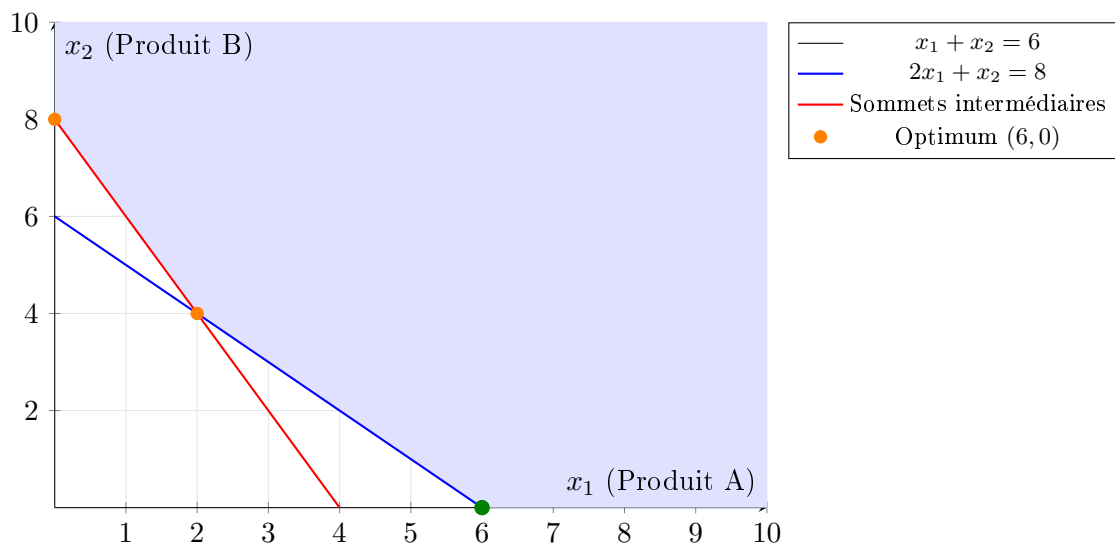
Exercice 2 Minimisation (région non bornée)

Un éleveur du Trarza compose un approvisionnement : A (x_1) et B (x_2). Minimiser $W = 2x_1 + 3x_2$.

$$\min W = 2x_1 + 3x_2 \quad \text{sous} \quad \begin{cases} x_1 + x_2 \geq 6 \\ 2x_1 + x_2 \geq 8 \\ x_1, x_2 \geq 0 \end{cases}$$

2.1 Domaine admissible (ouvert / non borné)

Intersection de $x_1 + x_2 = 6$ et $2x_1 + x_2 = 8$: $x_1 = 2$, $x_2 = 4$. Les points $(4, 0)$ et $(0, 6)$ sont **rejetés** (ils violent respectivement la contrainte 1 et la contrainte 2). Les sommets valides sont $(6, 0)$, $(2, 4)$, $(0, 8)$, avec une région non bornée vers le haut.



2.2 Optimum par la méthode de la droite d'isocoût

Sommet	$W = 2x_1 + 3x_2$
$(6, 0)$	12
$(2, 4)$	16
$(0, 8)$	24

En faisant glisser la droite d'isocoût $2x_1 + 3x_2 = c$ vers l'origine, le dernier point de contact avec la région admissible est $(6, 0)$.

Optimum : $x_1 = 6, x_2 = 0, W^* = 12$. Seule la contrainte $x_1 + x_2 \geq 6$ est saturée. Important : bien que la région soit non bornée, la *minimisation* admet ici une solution finie — ce ne serait pas le cas pour une maximisation sur cette même région.

2.3 Vérification avec PuLP

```

1  import pulp
2
3  prob = pulp.LpProblem("Minimisation_Cout_Approvisionnement", pulp.LpMinimize)
4
5  x1 = pulp.LpVariable("x1_ProduitA", lowBound=0)
6  x2 = pulp.LpVariable("x2_ProduitB", lowBound=0)
7
8  prob += 2 * x1 + 3 * x2, "Cout_total"
9  prob += x1 + x2 >= 6, "Contrainte_1"
10 prob += 2 * x1 + x2 >= 8, "Contrainte_2"
11
12 prob.solve()
13
14 print("Statut :", pulp.LpStatus[prob.status])
15 print(f"x1 = {x1.varValue}, x2 = {x2.varValue}")
16 print(f"W = {pulp.value(prob.objective)}")

```

Sortie attendue : Statut: Optimal, $x_1 = 6.0, x_2 = 0.0, W = 12.0$ — confirme la solution graphique.

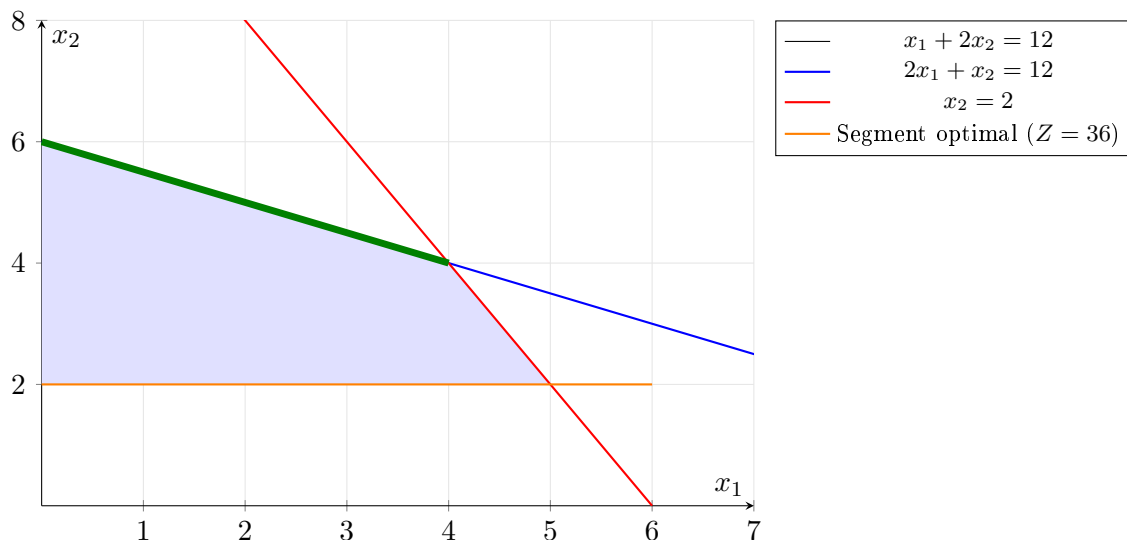
Exercice 3 Solutions multiples

Soit $\max Z = 3x_1 + 6x_2$ sous $x_1 + 2x_2 \leq 12, 2x_1 + x_2 \leq 12, x_2 \geq 2$. Démontrez graphiquement l'infinité de solutions optimales.

3.1 Sommets et évaluation

Les sommets de la région admissible sont $(0, 2), (5, 2), (4, 4), (0, 6)$.

Sommet	$Z = 3x_1 + 6x_2$
$(0, 2)$	12
$(5, 2)$	27
$(4, 4)$	36
$(0, 6)$	36



3.2 Démonstration de l'infinité de solutions

On remarque que

$$Z = 3x_1 + 6x_2 = 3(x_1 + 2x_2).$$

Or la contrainte active entre $(4, 4)$ et $(0, 6)$ est précisément $x_1 + 2x_2 = 12$: **la droite d'isoprofit est parallèle à cette contrainte**. Donc pour tout point du segment $[(4, 4); (0, 6)]$, vérifiant $x_1 + 2x_2 = 12$, on a $Z = 3 \times 12 = 36$.

Conclusion : tous les points du segment reliant $(4, 4)$ et $(0, 6)$ sont optimaux : il existe une **infinité de solutions optimales**, toutes de valeur $Z^* = 36$.

3.3 Vérification avec PuLP

```

1 import pulp
2
3 prob = pulp.LpProblem("Solutions_Multiples", pulp.LpMaximize)
4 x1 = pulp.LpVariable("x1", lowBound=0)
5 x2 = pulp.LpVariable("x2", lowBound=2)
6
7 prob += 3 * x1 + 6 * x2, "Z"
8 prob += x1 + 2 * x2 <= 12, "C1"
9 prob += 2 * x1 + x2 <= 12, "C2"
10
11 prob.solve()
12 print("Statut :", pulp.LpStatus[prob.status])
13 print(f"x1 = {x1.varValue}, x2 = {x2.varValue}, Z = {pulp.value(prob.objective)}")

```

Remarque : PuLP/CBC ne renvoie qu'un *seul* sommet optimal (par exemple $(0, 6)$ ou $(4, 4)$ selon l'algorithme), sans signaler nativement l'existence d'une infinité d'alternatives. La détection formelle nécessite de comparer le vecteur objectif à la normale de la contrainte active, comme fait ci-dessus.

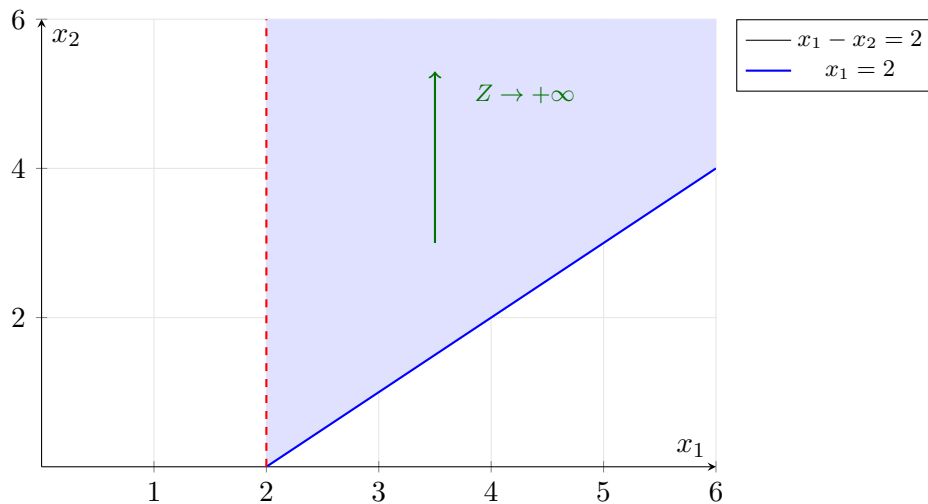
Exercice 4 Domaine non borné

Soit $\max Z = 4x_1 + 2x_2$ sous $x_1 - x_2 \leq 2$ et $2x_1 \geq 4$. Concluez sur la nature de la solution. Quelle erreur PuLP renverrait-il ?

4.1 Analyse

Le domaine est défini par $x_1 \geq 2$, $x_2 \geq x_1 - 2$, $x_2 \geq 0$.

Pour $x_1 = 2$ fixé, la contrainte $x_1 - x_2 \leq 2$ devient $2 - x_2 \leq 2 \Leftrightarrow x_2 \geq 0$, **toujours vérifiée**. On peut donc augmenter x_2 indéfiniment tout en restant dans le domaine admissible. Comme $Z = 4x_1 + 2x_2$ croît strictement avec x_2 , on a $Z \rightarrow +\infty$.



Conclusion : la solution est **non bornée** (« unbounded »). C'est le cas pathologique inverse de l'Exercice 2 : ici, une *maximisation* sur une région non bornée dans la direction où l'objectif croît n'admet pas de solution finie.

4.2 Erreur renvoyée par PuLP

```

1  import pulp
2
3  prob = pulp.LpProblem("Domaine_Non_Borne", pulp.LpMaximize)
4  x1 = pulp.LpVariable("x1", lowBound=0)
5  x2 = pulp.LpVariable("x2", lowBound=0)
6
7  prob += 4 * x1 + 2 * x2, "Z"
8  prob += x1 - x2 <= 2, "C1"
9  prob += 2 * x1 >= 4, "C2"
10
11 prob.solve()
12 print("Statut :", pulp.LpStatus[prob.status])

```

Sortie attendue :

Statut : Unbounded

Le solveur CBC détecte que la fonction objectif peut croître indéfiniment sans violer aucune contrainte, et retourne le code de statut `pulp.LpStatusUnbounded` (−3) au lieu de fournir des valeurs numériques pour x_1 et x_2 .

Exercice 5 Sensibilité

Pour $\max Z = C_1x_1 + 3x_2$ sous $2x_1 + x_2 \leq 10$ et $x_1 + 2x_2 \leq 8$, déterminez l'intervalle de C_1 qui maintient le sommet optimal inchangé.

5.1 Sommets

Les sommets sont $(0, 0)$, $(5, 0)$, $(4, 2)$, $(0, 4)$; le sommet $(4, 2)$ correspond à l'intersection des deux contraintes actives.

5.2 Condition de stabilité du sommet $(4, 2)$

La droite d'isoprofit $C_1x_1 + 3x_2 = Z$ a pour pente $-C_1/3$. Le sommet $(4, 2)$ reste optimal tant que cette pente reste comprise entre les pentes des deux contraintes actives :

- $2x_1 + x_2 = 10$: pente -2
- $x_1 + 2x_2 = 8$: pente $-1/2$

$$-2 \leq -\frac{C_1}{3} \leq -\frac{1}{2} \iff 1,5 \leq C_1 \leq 6$$

Vérification aux bornes :

- $C_1 = 6$: $Z(4, 2) = 6(4) + 3(2) = 30$ et $Z(5, 0) = 6(5) = 30$ — égalité, bascule vers $(5, 0)$ au-delà.
- $C_1 = 1,5$: $Z(4, 2) = 1,5(4) + 3(2) = 12$ et $Z(0, 4) = 3(4) = 12$ — égalité, bascule vers $(0, 4)$ en-deçà.

Résultat : pour $C_1 \in [1,5; 6]$, le sommet optimal $(4, 2)$ reste inchangé. En dehors de cet intervalle, l'optimum se déplace vers $(5, 0)$ (si $C_1 > 6$) ou $(0, 4)$ (si $C_1 < 1,5$).

5.3 Vérification avec PuLP

```

1 import pulp
2
3 def resoudre(c1_val):
4     prob = pulp.LpProblem("Sensibilite_C1", pulp.LpMaximize)
5     x1 = pulp.LpVariable("x1", lowBound=0)
6     x2 = pulp.LpVariable("x2", lowBound=0)
7     prob += c1_val * x1 + 3 * x2, "Z"
8     prob += 2 * x1 + x2 <= 10, "C1"
9     prob += x1 + 2 * x2 <= 8, "C2"
10    prob.solve()
11    return x1.varValue, x2.varValue, pulp.value(prob.objective)
12
13 for c1_test in [1.0, 1.5, 3.0, 6.0, 7.0]:
14     x1_val, x2_val, z_val = resoudre(c1_test)
15     print(f"C1={c1_test} -> x1={x1_val}, x2={x2_val}, Z={z_val}")

```

Résultats attendus : pour $C_1 = 1,0$ et $C_1 = 7,0$ (hors intervalle), le sommet optimal bascule respectivement vers $(0, 4)$ et $(5, 0)$. Pour $C_1 = 1,5$ et $C_1 = 6,0$ (bornes), il y a égalité entre deux sommets. Pour $C_1 = 3,0$ (dans l'intervalle), le sommet $(4, 2)$ est confirmé optimal avec $Z = 18$.

Exercice 6 Planification de production multi-sites

Une entreprise possède deux usines (U_1, U_2) et trois entrepôts (E_1, E_2, E_3).

	E_1	E_2	E_3	Capacité
U_1	5	2	4	500
U_2	3	4	6	700
Demande	300	400	350	

6.1 Modélisation

Variable : $x_{i,j}$ = quantité expédiée de l'usine $i \in \{U_1, U_2\}$ vers l'entrepôt $j \in \{E_1, E_2, E_3\}$, avec $x_{i,j} \geq 0$.

Remarque : l'offre totale ($500 + 700 = 1200$) est supérieure à la demande totale ($300 + 400 + 350 = 1050$). Ce problème de transport est donc **non équilibré** : les contraintes de capacité sont des inégalités ($\leq$), et les contraintes de demande sont des égalités ($=$), puisque toute la demande doit être satisfaite mais pas nécessairement toute la capacité de production.

6.2 Programme linéaire

$$\min C = 5x_{U_1, E_1} + 2x_{U_1, E_2} + 4x_{U_1, E_3} + 3x_{U_2, E_1} + 4x_{U_2, E_2} + 6x_{U_2, E_3}$$

Contraintes de capacité (offre) :

$$x_{U_1, E_1} + x_{U_1, E_2} + x_{U_1, E_3} \leq 500 \quad x_{U_2, E_1} + x_{U_2, E_2} + x_{U_2, E_3} \leq 700$$

Contraintes de demande :

$$x_{U_1, E_1} + x_{U_2, E_1} = 300 \quad x_{U_1, E_2} + x_{U_2, E_2} = 400 \quad x_{U_1, E_3} + x_{U_2, E_3} = 350$$

$$x_{i,j} \geq 0 \quad \forall i, j$$

6.3 Implémentation Python (PuLP, variables indexées)

```

1  import pulp
2
3  usines = ["U1", "U2"]
4  entrepots = ["E1", "E2", "E3"]
5
6  capacite = {"U1": 500, "U2": 700}
7  demande = {"E1": 300, "E2": 400, "E3": 350}
8
9  cout = {
10      ("U1", "E1"): 5, ("U1", "E2"): 2, ("U1", "E3"): 4,
11      ("U2", "E1"): 3, ("U2", "E2"): 4, ("U2", "E3"): 6,
12  }
13
14  prob = pulp.LpProblem("Transport_Multi_Sites", pulp.LpMinimize)
15
16  x = pulp.LpVariable.dicts(
17      "x",
18      [(i, j) for i in usines for j in entrepots],
19      lowBound=0, cat="Continuous"
20  )
21
22  prob += pulp.lpSum(cout[i, j] * x[i, j] for i in usines for j in entrepots), "
      Cout_total "
23
24  for i in usines:
25      prob += pulp.lpSum(x[i, j] for j in entrepots) <= capacite[i], f"Capacite_{i}
      )"
26
27  for j in entrepots:
28      prob += pulp.lpSum(x[i, j] for i in usines) == demande[j], f"Demande_{j}"
29
30  prob.solve()
31
32  print("Statut :", pulp.LpStatus[prob.status])
33  print(f"Cout total minimal = {pulp.value(prob.objective)}")

```



```

34 for i in usines:
35     for j in entrepots:
36         if x[i, j].varValue > 0:
37             print(f"  {i} -> {j} : {x[i, j].varValue:.0f} unites")

```

6.4 Résultat

Le programme a été validé par calcul numérique (solveur HiGHS) :

	E_1	E_2	E_3	Total
U_1	0	400	100	500
U_2	300	0	250	550

Plan de transport optimal :

- $U_1 \rightarrow E_2$: 400 unités (coût unitaire 2)
- $U_1 \rightarrow E_3$: 100 unités (coût unitaire 4)
- $U_2 \rightarrow E_1$: 300 unités (coût unitaire 3)
- $U_2 \rightarrow E_3$: 250 unités (coût unitaire 6)

Coût total minimal : $C^* = 3600$ MRU. La capacité de U_1 est entièrement utilisée (500/500) tandis que U_2 conserve 150 unités de capacité non utilisée (550/700), cohérent avec le fait que l'offre totale excède la demande totale.